Embedded fundamentals and TinyGo

Microcontrollers, peripherals, and Go on bare metal

Dinu-Ștefan Rusu
FILS, Universitatea Politehnica București · Spring 2027

TODAY

What you should leave with

Read the first page of a datasheet

Core, flash, SRAM, peripherals and clocks, and what those numbers forbid in your code.

Pick an execution model

A super loop with interrupts and timers, or an RTOS with tasks. Know which one your project needs.

Drive a peripheral

GPIO, ADC and PWM, and the difference between I2C, SPI and UART when you pick a sensor.

Ship a TinyGo program

Build, flash, read the serial console, and keep the device asleep between readings.

PART 1 · ANATOMY

Four orders of magnitude down

What a microcontroller is made of, and what it does not have

ANATOMY

The constraint jump: phone to microcontroller

	PHONE	MICROCONTROLLER
RAM	8–16 GB	roughly 520 KB of SRAM
Storage	128–512 GB	roughly 4 MB of flash
Power	roughly 5 W peak	milliwatts, or a coin cell
OS	Android or iOS	a small kernel, or nothing
Your code	Dart, ahead of time	Go, via TinyGo and LLVM

This is the far right of the device spectrum from Lecture 1. Every habit from that lecture applies, with no headroom left to absorb a mistake.

ANATOMY

What is inside a microcontroller

Core

One CPU, a Cortex-M or a RISC-V part. No memory management unit, so no processes.

Flash

Holds your program and its constants. Code runs from it directly.

SRAM

Globals, stacks and the heap share one small block. You run out of this first.

Peripherals

GPIO, timers, ADC, PWM, I2C, SPI, UART. Hardware blocks driven by registers.

Clocks

An internal oscillator or a crystal, divided out to the core and each peripheral.

Power domains

Parts of the chip switch off independently. That is what makes deep sleep work.

ANATOMY

The memory map

REGION	WHAT LIVES THERE	WHY YOU CARE
Vector table	Handler addresses	The first thing the core reads
Flash	Code and constants	Read-only at run time
SRAM	Globals, stack and heap	One block, shared by all three
Peripherals	Control and data registers	One store turns a pin on
Core private	Interrupt controller, timer	Fixed by the core design

Everything lives in one flat address space. There is no protection, so a wild pointer can overwrite a peripheral register as easily as a variable.

ANATOMY

What half a megabyte of RAM means

Statics are cheap

A fixed-size array decided at compile time never fails to allocate.

The heap is a risk

Allocation can fail, and no operating system will restart you when it does.

The stack is small

Deep recursion and large local buffers corrupt memory silently.

Buffers, not strings

Building strings allocates. Reuse one byte slice and write into it.

Parse incrementally

Decoding a whole JSON document needs it all in RAM. Read field by field.

Collection still happens

The collector is small and simple. Allocating less is the only real fix.

PART 2 · EXECUTION

Bare metal or a kernel

The super loop, interrupts, timers, and what an RTOS adds

The super loop

```
func main() {
    led := machine.LED
    led.Configure(machine.PinConfig{
        Mode: machine.PinOutput})

    for {               // this never
    returns
        led.High()
        time.Sleep(200 * time.Millisecond)
        led.Low()
        time.Sleep(800 * time.Millisecond)
    }
}
```

There is nothing to return to.
If `main` exits, the board stops or resets. The loop is the program. Every step in the loop delays every other step. A sensor read that blocks for 100 ms is 100 ms in which nothing else happens.

Rule of thumb

A super loop is right while one slow step is still acceptable.

Interrupts and timers

```
var pressed volatile.Register8

button.Configure(machine.PinConfig{
    Mode: machine.PinInputPullup})

button.SetInterrupt(machine.PinFalling,
    func(p machine.Pin) {
        pressed.Set(1)   // set a flag
    })
```

The handler contract.
Run for microseconds. Set a flag or push into a buffer. Never sleep, never allocate, never print.
A hardware timer is the same idea on a schedule: the peripheral counts on its own and interrupts you when it wraps. That is how you sample at a fixed rate without a blocking sleep.

An interrupt steals the CPU from your loop. Whatever the handler does, the main loop pays for it in latency. Keep handlers short and do the work in the loop.

Bare metal versus an RTOS

	SUPER LOOP	REAL-TIME KERNEL
Scheduling	You write it, by hand	Preemptive, by task priority
A blocking call	Stalls everything	Blocks one task, others keep running
Timing	Best effort	Deadlines you can reason about
RAM cost	None beyond your code	A stack per task, plus the kernel
Failure mode	One slow step delays all	Races and priority inversion
Right when	One sensor, one radio	Several rates and several deadlines

Most student projects belong in the left column. A TinyGo program with goroutines sits between the two: cooperative scheduling, no priorities, and no kernel to configure.

FreeRTOS in one slide

Tasks

A function with its own stack and a priority, switched by the kernel.

Priorities

The highest ready priority runs. A task that never blocks starves the rest.

Queues

The safe way to move data between tasks and out of a handler.

Mutexes

Protect a shared peripheral, such as one bus used by two tasks.

Semaphores

A handler signals, a task wakes. This is how work leaves an interrupt.

Ticks

The kernel timer defines every delay, timeout and scheduling step.

ESP-IDF ships FreeRTOS. From TinyGo, goroutines are your task model.

PART 3 · PERIPHERALS

Talking to the outside world

GPIO, ADC, PWM, and choosing between I2C, SPI and UART

The peripherals you will actually use

GPIO

One pin, high or low. Buttons, relays, LEDs, and the chip-select line of an SPI device.

ADC

Turns a voltage into an integer. Potentiometers, light sensors, analog microphones.

PWM

A square wave with an adjustable duty cycle. LED brightness, motor speed, servo angle.

I2C

Two shared wires, many addressed devices. Almost every small digital sensor.

SPI

Four wires, one select line per device, megahertz speeds. Displays, flash chips, radios.

UART

Two wires, point to point, a fixed baud rate. Serial logs, GPS modules, modems.

GPIO: the simplest peripheral

```
led := machine.LED
led.Configure(machine.PinConfig{
    Mode: machine.PinOutput})
led.Set(true)

btn := machine.GP15
btn.Configure(machine.PinConfig{
    Mode: machine.PinInputPullup})

down := !btn.Get()  // pull-up: low
```

Wiring, in words.
The LED goes from the pin through a resistor to ground. The button connects the pin to ground, and the internal pull-up holds the pin high until it is pressed.

⚠ Bouncing

A contact makes several transitions in a few milliseconds. Ignore later edges for roughly 20 ms.

ADC: turning a voltage into a number

```
machine.InitADC()

sensor := machine.ADC{Pin: machine.ADC0}
sensor.Configure(machine.ADCConfig{})

raw := sensor.Get()      // 0 .. 65535
// TinyGo scales every board to 16 bits,
// whatever the hardware resolution is.
volts := float32(raw) * 3.3 / 65535.0
```

Wiring, in words.
A resistive sensor forms a divider between the supply and ground. Its middle point goes to the ADC pin. Readings jitter. Average several samples, or you will show noise and call it a measurement.

Reference

The number means nothing until you know the reference voltage.

PWM: an average voltage from a square wave

```
pwm := machine.PWM0
// Period is in nanoseconds: 500 Hz here.
pwm.Configure(machine.PWMConfig{
    Period: 1e9 / 500,
})

ch, _ := pwm.Channel(machine.GP0)
pwm.Set(ch, pwm.Top()/4)  // quarter duty
```

The pin is still only high or low. The peripheral toggles it fast enough that the load sees the average. An LED dims, a motor slows. `Top()` is the counter maximum for the configured period, so a duty cycle is a fraction of it.

⚠ Which timer

PWM units are shared between pins. Two pins on the same unit share one period.

Picking a bus: I2C, SPI, or UART

BUS	WIRES	SPEED	DEVICES	USE IT FOR
I2C	2, shared	100–400 kHz	many, by address	small digital sensors
SPI	3, plus 1 select	megahertz	a few	displays, flash, radios
UART	2, point to point	a fixed baud	one	serial logs, GPS, modems

I2C addresses devices in software, so two wires reach a dozen sensors. SPI is faster but spends a select pin per device. UART has no addressing, so it joins exactly two parties.

Read the sensor’s datasheet before you buy it. The bus decides your wiring, your pin budget, and whether a driver already exists.

Wiring an I2C sensor

Four connections

Supply, ground, and the clock and data lines to the two I2C pins of your board.

Two pull-up resistors

Both lines are open drain. Most breakout boards already carry the pair.

One address

Seven bits, fixed by the maker. Two identical sensors need different addresses.

One voltage

A 3.3 V board and a 5 V sensor need a level shifter, or the board dies.

If nothing answers, scan the bus first. A device that never acknowledges its address is a wiring problem.

PART 4 · TINYGO

Go, compiled for bare metal

The toolchain, the machine package, and the parts of Go you give up

TinyGo: Go, compiled for bare metal

TinyGo reads ordinary Go, emits LLVM intermediate code, and has LLVM produce machine code for Cortex-M, RISC-V, AVR and WebAssembly. It is a different compiler for the same language.

WHAT YOU KEEP

Goroutines and channels on a chip with no operating system.

Binaries measured in tens of kilobytes.

The machine package: one hardware API for every supported board.

One language shared with the Go backend of Lecture 10.

WHAT YOU GIVE UP

A reduced runtime: cooperative scheduling, a simple collector.

Limited reflection, so reflection-heavy packages will not build.

A subset of the standard library, with no full network stack.

Some Go packages do not compile for a microcontroller.

Which board to buy

Do not start with the original ESP32.

The classic Xtensa ESP32, in most tutorials and starter kits, has minimal TinyGo support and no Wi-Fi or Bluetooth.

Buy one of these instead

ESP32-C3 or ESP32-S3: RISC-V parts, with Wi-Fi through the espradio package since TinyGo 0.41.

Raspberry Pi Pico, RP2040 or RP2350: first-tier support, best documented.

```
# One flag is the difference:
tinygo flash -target=pico .
tinygo flash -target=esp32c3 .

# What does this board have?
tinygo targets
```

No board yet

Wokwi runs the same binary in a browser.

Build, flash, monitor

```
# Compile to a file you can inspect or
    copy.
tinygo build -target=pico -o blink.uf2 \
    ./cmd/blink

# Compile and write it to the board.
tinygo flash -target=pico ./cmd/blink

# Read what the running board prints.
tinygo monitor

# How big did it get?
tinygo build -size short -target=pico \
    -o blink.uf2 ./cmd/blink
```

Three commands, one loop.
Edit, flash, monitor. No hot reload, and no debugger attached by default.
The size report prints the flash and RAM the binary claims. Watch the RAM column: it is the one that runs out.

⚠ Bootloader mode

Many boards need a button held during reset to accept a flash.

The machine package: a pin, a sensor, a loop

```
func main() {
    led := machine.LED
    led.Configure(machine.PinConfig{
        Mode: machine.PinOutput})
    machine.InitADC()
    s := machine.ADC{Pin: machine.ADC0}
    s.Configure(machine.ADCConfig{})
    for {
        led.Set(true) // proof of life
        println("adc:", s.Get())
        time.Sleep(2 * time.Second)
    }
}
```

One API, every board.

`machine.LED` and `machine.ADC0` are aliases resolved per target, so this source flashes to a Pico and an ESP32-C3 unchanged.

`println` writes to the serial port. That is why the monitor is your debugger, and why an LED in the loop is the cheapest proof that the loop still runs.

Reading a sensor over I2C

```
i2c := machine.I2C0
i2c.Configure(machine.I2CConfig{
    Frequency: 400 * machine.KHz})

w := []byte{0x00}   // register to read
r := make([]byte, 2)

if err := i2c.Tx(0x48, w, r); err != nil {
    println("i2c:", err.Error())
}
raw := int16(uint16(r[0])<<8 |
    uint16(r[1]))
```

Write the register, then read the bytes.

Almost every I2C sensor works this way, and `Tx` does both halves in one transaction.

Do not write this by hand. The drivers repository carries a few hundred sensors, each hiding the register map behind a typed reader.

Lab 2

Pick a sensor that has a driver.

Serial is your debugger

```
uart := machine.UART0
uart.Configure(machine.UARTConfig{
    BaudRate: 115200,
})
uart.Write([]byte("boot ok\r\n"))

// println goes to the same place and
// costs less code than fmt.Sprintf.
println("seq:", seq, "raw:", raw)
```

Print one line per loop, with a counter.
A counter that stops means the loop died. A counter that jumps back to zero means the board reset.
`fmt` pulls in a lot of code and allocates. Prefer `println` and plain integers. A stepping debugger needs a probe over SWD or JTAG. Not what the lab uses.

Every embedded bug looks the same at first: the board went quiet. A sequence number in every line tells you whether it stopped, reset, or lost its connection.

The parts of Go you give up

Reflection

Partial. Packages built on it, such as the standard JSON encoder, will not build.

The scheduler

Cooperative. A goroutine that never blocks starves the others.

The collector

Small and simple. Allocate little and it will not bother you.

The net package

Absent on most targets. Networking comes from packages tied to the radio.

The standard library

A subset. Check a package builds for your target early.

cgo and plugins

Not the tools here. Assume one statically linked binary.

None of this is a language change. Each is a runtime or library limit.

Goroutines on a chip

```
readings := make(chan uint16, 4)

go func() {
    for {
        readings <- sensor.Get()
        time.Sleep(time.Second)
    }
}()

for r := range readings {
    println("adc:", r)   // publish here
}
```

The same producer and consumer as Lecture 2.

One goroutine samples on a schedule, the main loop drains the channel. Each goroutine costs a stack of a few hundred bytes. Affordable for a handful, not for hundreds.

⚠ Cooperative

A goroutine yields only when it blocks or sleeps. A busy loop freezes everything.

PART 5 · POWER

Wake, send, sleep

Sleep modes, wake sources, the watchdog, and what Lab 1 needs

POWER

Where the milliamps go

radio

dominates the budget, not the processor

4

orders of magnitude between deep sleep and transmitting

duty

cycle, not clock speed, decides battery life

Average current is roughly the active current times the active time, plus the sleep current times the sleep time, all divided by the period. Shorten the active window or lengthen the period, and everything else is noise.

The question is not how fast the code runs. It is what fraction of the time the radio is powered.

POWER

Sleep modes, wake sources, and the watchdog

MODE	CPU	RADIO	RAM	WAKES ON
Run	running	on	kept	it is already awake
Idle	halted	on	kept	any interrupt, in microseconds
Light sleep	halted	off	kept	a timer or a pin, in a millisecond
Deep sleep	off	off	partly	a timer or a chosen pin, then a reset

Deep sleep restarts the program from the top, so surviving state goes to retained memory or flash. The watchdog is a hardware countdown the loop resets each pass.

Feed the watchdog in the main loop and nowhere else. Feeding it from an interrupt keeps the timer happy while the program is already stuck.

POWER

Wake, send, sleep

```
for {
    buf = append(buf, sample())
    if len(buf) >= 6 {
        connect()       // the expensive
        publish(buf)    // part of the loop
        disconnect()
        buf = buf[:0]
    }
    time.Sleep(30 * time.Second)
}
```

Staying connected loses to waking up.
A keep-alive powers the radio forever to save a handshake you pay once. Batch readings before you spend a connection. Six in one publish cost one sixth of the radio time. Accept the reconnect unless commands must arrive fast.

This is the same optimization as batching network calls on a phone. There it saves an afternoon. On a coin cell it decides between a week and a year.

LAB 1

Set up before Lab 1

Install the toolchain

Install Go, then TinyGo. Run the targets command and find your board in the list.

Have a board, or a simulator

A Raspberry Pi Pico, or an ESP32-C3 or S3. Without hardware, use Wokwi: the lab names the board.

Flash and monitor

Flash the blink example and read the serial console. If both work, the lab is about the sensor.

Run a broker

Install Mosquitto locally and verify publish and subscribe with the command-line clients.

Bring the team sheet filled in. Lab 1 also approves the project idea.

PROJECT

What the project device must do

A real device component

A TinyGo program on a supported board, or in the Wokwi simulator, that reads at least one sensor. Blink alone does not count.

One live link to the phone

The reading reaches the app over MQTT or over BLE, and the app shows it changing. Lecture 4 and Lecture 5 cover both.

Milestone 3

By the third project session, the device streams a reading to a serial console. That is this lecture plus one sensor.

One measured number

Energy, latency, payload size or frame time, with the method you used. The device side is the easiest place to find one.

RECAP

Three things to remember

1. A microcontroller is one flat address space with no protection. SRAM is the resource you run out of, and a peripheral is just an address you write to.
2. Pick the simplest execution model that meets your deadlines. A super loop with interrupts and timers carries most projects. An RTOS buys priorities and costs RAM.
3. TinyGo keeps the language and the concurrency model. It gives up reflection, most of the network stack, and a large part of the library ecosystem.

FURTHER READING

tinygo.org board support table · the drivers repository · tinygo.org tutorials · wokwi.com

Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel